BCS702 - Parallel Computing

Important Questions - Part 2

Unit 3 (MPI), Unit 4 (OpenMP), Unit 5 (CUDA) - 10 Marks Each (VTU Format)

Q. No.	Unit	Question
Q.1	Unit 3	Explain parallelising a sorting algorithm using odd even transposition sort.
Q.2	Unit 4	Explain parallelising a trapezoidal rule using OpenMP.
Q.3	Unit 5	Explain the architectures used in CUDA. Explain the role of threads, blocks and grids.
Q.4	Unit 5	Explain trapezoidal rule using CUDA.
Q.5	Unit 5	Explain the history of CUDA.
Q.6	Unit 5	Write a CUDA C program to implement vector addition using parallel threads.

Writing tip: write the question, a short **introduction**, then the headings given here. Draw the **diagrams**, write **syntax / code** neatly and end with a **conclusion**. Underline the **bold keywords**. (The vector addition question appeared twice in the list; it is answered once, as Q.6.)

UNIT 3 - Distributed Memory Programming with MPI

Q. 1 Explain parallelising a sorting algorithm using odd even transposition sort. [10 Marks]

Introduction

Odd-even transposition sort is a variant of bubble sort that has considerably more opportunities for parallelism. The key idea is to **decouple the compare-swaps**. The algorithm is a sequence of **phases of two types**:

- **Even phase:** compare-swap the pairs `(a[0],a[1]), (a[2],a[3]), (a[4],a[5]), ...`
- **Odd phase:** compare-swap the pairs `(a[1],a[2]), (a[3],a[4]), (a[5],a[6]), ...`

Example: Start 5, 9, 4, 3. Even phase: (5,9) and (4,3) give 5, 9, 3, 4. Odd phase: (9,3) gives 5, 3, 9, 4. Even phase: (5,3), (9,4) give 3, 5, 4, 9. Odd phase: (5,4) gives **3, 4, 5, 9**.

Theorem: after **n phases** a list of n keys is sorted.

Serial algorithm

```
void Odd_even_sort(int a[], int n) {
    int phase, i, temp;
    for (phase = 0; phase < n; phase++)
        if (phase % 2 == 0) {                       /* Even phase */
            for (i = 1; i < n; i += 2)
                if (a[i-1] > a[i]) { temp = a[i]; a[i] = a[i-1]; a[i-1] = temp; }
        } else {                                    /* Odd phase */
            for (i = 1; i < n-1; i += 2)
                if (a[i] > a[i+1]) { temp = a[i]; a[i] = a[i+1]; a[i+1] = temp; }
        }
}
```

Parallelising with Foster's methodology

- **Tasks:** determine the value of `a[i]` at the end of phase `j`.
- **Communication:** the task for `a[i]` must communicate with the task for `a[i-1]` or `a[i+1]`; the value of `a[i]` after phase `j` is needed for phase `j+1`.
- **All compare-swaps in a single phase can happen simultaneously**, which is the source of parallelism.
- When `n = p`, process `i` exchanges `a[i]` with process `i-1` or `i+1`. This is impractical: we rarely have as many processes as keys, and **communication costs far more than a local compare-swap**.

Each process stores n/p keys (n divisible by p)

1. **Sort the local keys** first with a fast serial sort such as `qsort`.
2. In each phase a process exchanges **all its n/p keys** with its partner.
3. The lower-ranked process **keeps the smaller n/p keys**; the higher-ranked process **keeps the larger n/p keys**.
4. Processes without a partner in a phase are **idle**. After **p phases** (theorem) the list is sorted.

Example: p = 4 processes, n = 16 keys

Time	Process 0	Process 1	Process 2	Process 3
Start	15, 11, 9, 16	3, 14, 8, 7	4, 6, 12, 10	5, 2, 13, 1
After local sort	9, 11, 15, 16	3, 7, 8, 14	4, 6, 10, 12	1, 2, 5, 13
After phase 0	3, 7, 8, 9	11, 14, 15, 16	1, 2, 4, 5	6, 10, 12, 13
After phase 1	3, 7, 8, 9	1, 2, 4, 5	11, 14, 15, 16	6, 10, 12, 13

Time	Process 0	Process 1	Process 2	Process 3
After phase 2	1, 2, 3, 4	5, 7, 8, 9	6, 10, 11, 12	13, 14, 15, 16
After phase 3	1, 2, 3, 4	5, 6, 7, 8	9, 10, 11, 12	13, 14, 15, 16

Parallel algorithm

```
Sort local keys;
for (phase = 0; phase < comm_sz; phase++) {
    partner = Compute_partner(phase, my_rank);
    if (I'm not idle) {
        Send my keys to partner;
        Receive keys from partner;
        if (my_rank < partner) Keep smaller keys;
        else                   Keep larger keys;
    }
}
```

Computing the partner

```
if (phase % 2 == 0)                 /* Even phase */
    if (my_rank % 2 != 0) partner = my_rank - 1;   /* odd rank  */
    else                  partner = my_rank + 1;   /* even rank */
else                                /* Odd phase */
    if (my_rank % 2 != 0) partner = my_rank + 1;
    else                  partner = my_rank - 1;
if (partner == -1 || partner == comm_sz)
    partner = MPI_PROC_NULL;       /* idle: no communication takes place */
```

With `MPI_PROC_NULL` as source or destination, **no communication takes place**.

Safety in MPI programs

- If every process calls `MPI_Send` and then `MPI_Recv` to exchange with its partner, the program may **hang (deadlock)**. `MPI_Send` may either **buffer** the message or **block** until the matching receive starts; large messages usually block.
- A program that relies on MPI buffering is **unsafe**: it may run for small `n` but hang for large `n`.
- To test safety, replace `MPI_Send` by `MPI_Ssend` (synchronous: blocks until the matching receive starts). If the program does not hang, it was safe.
- Restructure so some processes **receive before sending**, or use `MPI_Sendrecv`: a blocking send and receive in one call, scheduled by MPI so the program will not hang.

```
MPI_Sendrecv(my_keys, n/comm_sz, MPI_INT, partner, 0,
             recv_keys, n/comm_sz, MPI_INT, partner, 0, comm, MPI_STATUS_IGNORE);
```

Keeping the smaller (or larger) keys: merge

Sorting the 2n/p keys is expensive. Since both lists are already sorted, **merge** them and stop once n/p keys are found. To get the largest n/p keys, merge backwards starting from `local_n - 1`.

```
void Merge_low(int my_keys[], int recv_keys[], int temp_keys[], int local_n) {
    int m_i = 0, r_i = 0, t_i = 0;
    while (t_i < local_n) {
        if (my_keys[m_i] <= recv_keys[r_i]) temp_keys[t_i++] = my_keys[m_i++];
        else                                temp_keys[t_i++] = recv_keys[r_i++];
    }
    for (m_i = 0; m_i < local_n; m_i++) my_keys[m_i] = temp_keys[m_i];
}
```

Conclusion

Sort local keys, then in `p` phases exchange keys with the partner using `MPI_Sendrecv` and keep the smaller or larger half by merging.

UNIT 4 - Shared-Memory Programming with OpenMP

Q. 2 Explain parallelising a trapezoidal rule using OpenMP. [10 Marks]

Introduction

The **trapezoidal rule** estimates the area under `y = f(x)` between `a` and `b` by dividing `[a, b]` into `n` equal subintervals and approximating the area over each by a **trapezoid**. With `h = (b - a)/n` and `x_i = a + i*h`: **Area = h [f(x0)/2 + f(x1) + ... + f(x(n-1)) + f(xn)/2]**

```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) { x_i = a + i*h; approx += f(x_i); }
approx = h*approx;                              /* serial code */
```

A. First OpenMP version (parallel + critical)

Using **Foster's methodology**: tasks are (a) computing the area of individual trapezoids and (b) adding the areas. There is no communication among the (a) tasks, but each communicates with (b). There are many more trapezoids than cores, so **a contiguous block of trapezoids is assigned to each thread**; each thread applies the serial rule to its own subinterval.

```
void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
    double global_result = 0.0;
    double a, b;  int n, thread_count;
    thread_count = strtol(argv[1], NULL, 10);
    printf("Enter a, b, and n\n");
    scanf("%lf %lf %d", &a, &b, &n);
#   pragma omp parallel num_threads(thread_count)
    Trap(a, b, n, &global_result);
    printf("With n = %d trapezoids, our estimate\n", n);
    printf("of the integral from %f to %f = %.14e\n", a, b, global_result);
    return 0;
}

void Trap(double a, double b, int n, double* global_result_p) {
    double h, x, my_result, local_a, local_b;
    int i, local_n;
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();

    h = (b-a)/n;
    local_n = n/thread_count;               /* trapezoids per thread */
    local_a = a + my_rank*local_n*h;        /* left end point        */
    local_b = local_a + local_n*h;          /* right end point       */
    my_result = (f(local_a) + f(local_b))/2.0;
    for (i = 1; i <= local_n-1; i++) {
        x = local_a + i*h;
        my_result += f(x);
    }
    my_result = my_result*h;
#   pragma omp critical
    *global_result_p += my_result;
}
```

- The `parallel` directive starts `thread_count` threads, each calling `Trap`. Each thread finds its rank and team size with `omp_get_thread_num()` and `omp_get_num_threads()`.
- Variables declared in `Trap` are **private** (thread's own stack). `a, b, n, global_result` declared in `main` before the parallel block are **shared**.
- **Race condition:** if two threads execute `global_result += my_result` together, one update overwrites the other (e.g. thread 0 stores 1, thread 1 stores 2 instead of 3). The statement is a **critical section**, protected by `#pragma omp critical` (mutual exclusion).

- `n` must be **evenly divisible** by `thread_count`; otherwise fewer than `n` trapezoids are used (e.g. n = 14, 4 threads: local_n = 3, only 12 trapezoids).

B. Using the reduction clause

Let each thread return its part from `double Local_trap(double a, double b, int n)`. Putting the **call** inside `critical` makes threads execute `Local_trap` one at a time (sequential, possibly slower). Better: use a private variable and keep only the addition critical, or use a **reduction clause**:

```
global_result = 0.0;
#  pragma omp parallel num_threads(thread_count) \
       reduction(+: global_result)
   global_result += Local_trap(a, b, n);
```

- A **reduction operator** is an associative binary operation (such as + or *); a **reduction** repeatedly applies it to a sequence of operands to get one result stored in the **reduction variable**.
- OpenMP creates a **private copy** of `global_result` per thread (initialised to the **identity value**: 0 for +, 1 for *), and combines the private copies into the shared variable in a critical section at the end.
- Syntax: `reduction(<operator>: <variable list>)`; operators in C: `+ * - & | ^ && ||`.
- Floating-point results may differ slightly with different numbers of threads because floating-point addition is not associative.

C. Using the parallel for directive (simplest)

```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) \
       reduction(+: approx)
   for (i = 1; i <= n-1; i++)
       approx += f(a + i*h);
approx = h*approx;
```

- `parallel for` forks the threads and **divides the iterations** among them (usually a block partition).
- `approx` must be a **reduction variable**, otherwise `approx += ...` is an unprotected critical section.
- The loop variable `i` is **private** by default.
- The loop must be in **canonical form** and have no loop-carried dependences.

Comparison

Version	How work is divided	How result is combined
A. `parallel` + `critical`	Programmer computes `local_a`, `local_b`, `local_n`	`critical` around `*global_result_p += my_result`
B. `reduction` clause	Programmer divides work; function returns local result	Private copies combined automatically by OpenMP
C. `parallel for`	System divides loop iterations	`reduction(+: approx)`

Conclusion

The trapezoidal rule is parallelised by giving each thread a block of trapezoids and adding the partial results safely, using `critical`, `reduction` or `parallel for` with `reduction`. The reduction clause avoids serialising the threads' work.

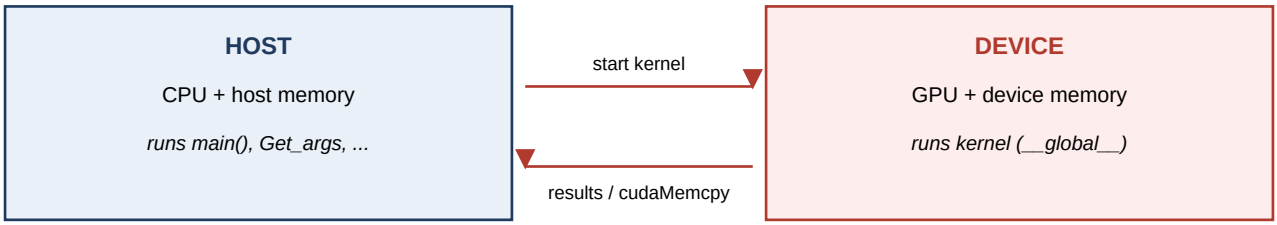
UNIT 5 - GPU Programming with CUDA

Q. 3 Explain the architectures used in CUDA. Explain the role of threads, blocks and grids. **[10 Marks]**

Introduction

CUDA is a software platform for writing GPGPU programs for **heterogeneous systems with an Nvidia GPU**. To program a GPU we first need its architecture.

1. Host and device

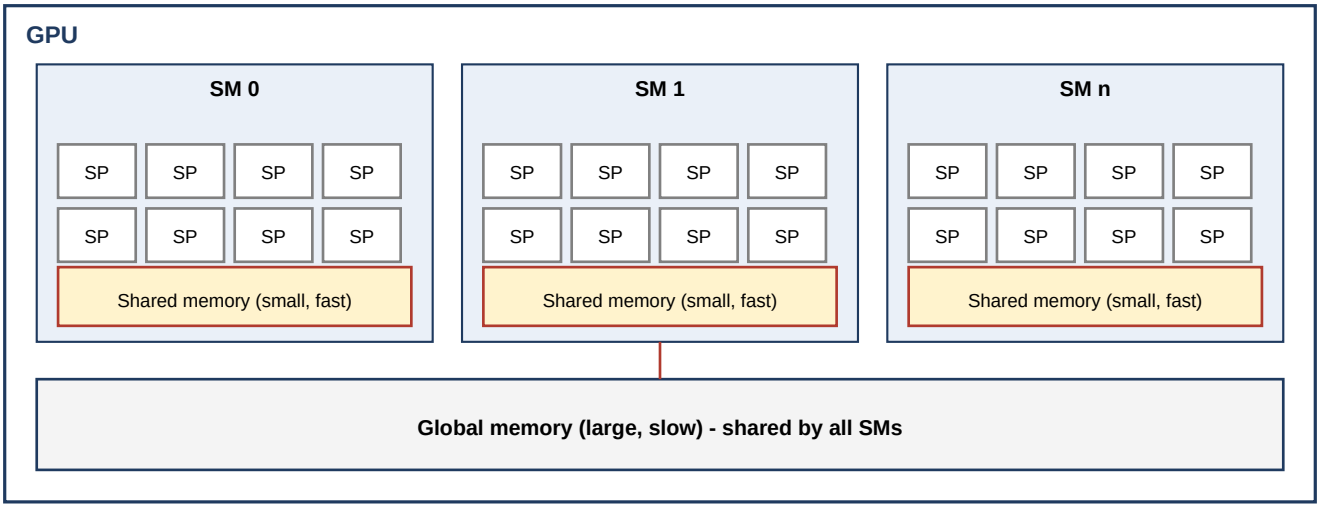


- **Host:** the CPU together with its memory. **Device:** the GPU together with its memory. They are usually **physically separate**.
- A program has **host code** (runs on CPU, starts in `main`) and **device code** (the **kernel**, runs on GPU). This is **heterogeneous computing**.
- Early systems needed explicit data transfers between host and device memory; devices with compute capability >= 3.0 provide **unified memory** (`cudaMallocManaged`).

2. SIMD / SIMT processors

- A conventional CPU is a **SISD** device. A GPU is composed of **SIMD** processors: a single **control unit** fetches an instruction and broadcasts it to many **datapaths**; each datapath either executes the instruction on its data or is **idle**.
- Nvidia uses the term **SIMT (Single Instruction Multiple Thread)**: threads executing the same instruction may not execute at the same moment, because some threads block on memory access while others proceed.

3. Streaming multiprocessors and streaming processors



- An Nvidia GPU is composed of **Streaming Multiprocessors (SMs)**. An SM has several control units and many datapaths, so it is like one or more SIMD processors.

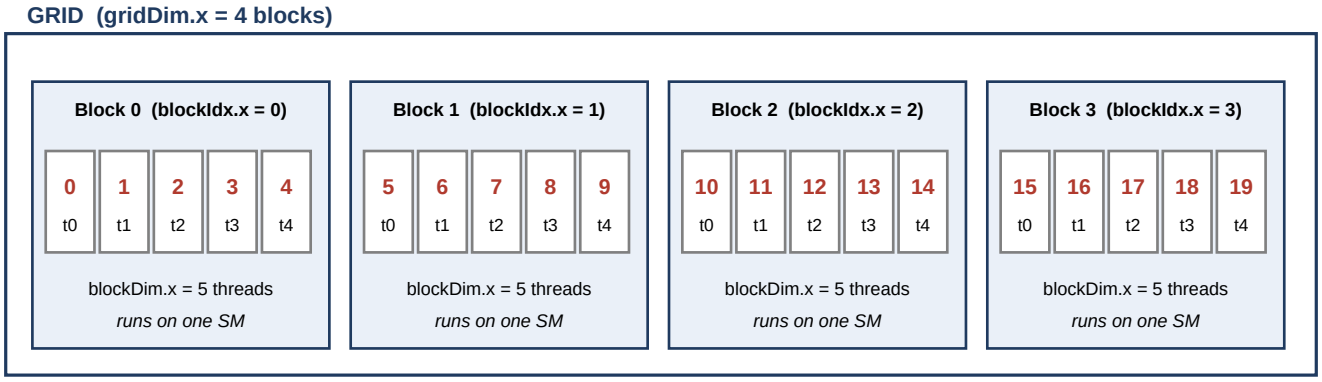
- The datapaths are called **Streaming Processors (SPs)** or cores. Example: a powerful Nvidia processor has **82 SMs x 128 SPs = 10,496 SPs**.
- SMs operate **asynchronously**: no penalty if two branches of an `if-else` run on different SMs.
- Each SM has a small, fast **shared memory** shared by its SPs; all SMs share a large but slow **global memory**.

4. Compute capability and architectures

Limits depend on the **compute capability** `a.b` of the GPU. For capability > 1: max **1024 threads per block**; max threads per SM is 1536 (2.b) and 2048 (> 2); max x or y dimension 1024, z dimension 64.

Architecture	Tesla	Fermi	Kepler	Maxwell	Pascal	Volta	Turing	Ampere
Compute capability	1.b	2.b	3.b	5.b	6.b	7.0	7.5	8.0

5. Role of threads, blocks and grids



- **Thread:** one copy of the kernel code, executed on an SP (SPMD). Its rank in its block is `threadIdx.x`.
- **Block:** a collection of threads that run on a **single SM**. Blocks must be independent (can run in any order, sequentially or in parallel) and have the same dimensions.
- **Grid:** the collection of thread blocks started by a kernel call.
- Kernel call: `Kernel<<<blk_ct, th_per_blk>>>(args)` - first value = number of blocks, second = threads per block. Total threads = `blk_ct * th_per_blk`.

Built-in variable	Meaning
`threadIdx`	Rank (index) of the thread in its block
`blockIdx`	Rank (index) of the block in the grid
`blockDim`	Size / shape of each block
`gridDim`	Size / shape of the grid

Each is a struct with fields `x, y, z`. Global rank of a thread: `rank = blockDim.x * blockIdx.x + threadIdx.x`. With `dim3` we can give 3-D shapes, e.g. a grid of 2 x 3 x 1 = 6 blocks, each 4 x 4 x 4 = 64 threads.

```
__global__ void Hello(void) { printf("Hello from thread %d in block %d\n", threadIdx.x, blockIdx.x); }
...
Hello<<<blk_ct, th_per_blk>>>();   cudaDeviceSynchronize();
```

Conclusion

CUDA uses a host-device model; the GPU consists of SMs with SPs that execute threads in SIMT fashion, with fast shared memory per SM and slow global memory. Threads are grouped into blocks (one SM each) and blocks into a grid.

UNIT 5 - GPU Programming with CUDA

Q. 4 Explain trapezoidal rule using CUDA. [10 Marks]

Introduction

The trapezoidal rule divides `[a, b]` into `n` equal subintervals of length `h = (b - a)/n`, `x_i = a + i*h`. The area of the i-th trapezoid is `(h/2)[f(x_i) + f(x_(i+1))]`, so the total is **h [(f(a)+f(b))/2 + f(x1) + f(x2) + ... + f(x(n-1))]**.

```
float Serial_trap(const float a, const float b, const int n) {
    float x, h = (b-a)/n;
    float trap = 0.5*(f(a) + f(b));
    for (int i = 1; i <= n-1; i++) { x = a + i*h; trap += f(x); }
    trap = trap*h;
    return trap;
}
```

Design using Foster's method

The main work is in the `for` loop. Tasks: evaluate `f(x_i)` and add `f(x_i)` into `trap`. The second depends on the first, so they are **aggregated**: **each CUDA thread carries out one iteration of the serial loop.** Each thread gets a unique rank `my_i = blockDim.x * blockIdx.x + threadIdx.x`, computes `my_x = a + my_i*h` and `my_trap = f(my_x)`.

Problems and their solutions

Problem	Solution
1. `h` and `trap` not initialised	Initialise `h` and `*trap_p = 0.5*(f(a)+f(b))` in the **host** before the kernel call
2. `my_i` can be too small or too large (loop runs 1 to n-1)	Test `if (0 < my_i && my_i < n)` in the kernel
3. `trap` must be **shared**; adding gives a **race condition**	Allocate one location with `cudaMallocManaged`, pass pointer `trap_p`; use `atomicAdd`
4. Kernels must return `void`	Return the result through the pointer `trap_p`
5. Multiply total by `h` at the end	Done on the host after the kernel: `*trap_p = h*(*trap_p)`

Program

```
__global__ void Dev_trap(const float a, const float b, const float h,
                         const int n, float* trap_p) {
    int my_i = blockDim.x * blockIdx.x + threadIdx.x;
    /* f(x_0) and f(x_n) were computed on the host. So compute f(x_1)..f(x_(n-1)) */
    if (0 < my_i && my_i < n) {
        float my_x = a + my_i*h;
        float my_trap = f(my_x);
        atomicAdd(trap_p, my_trap);
    }
}
/* Host code */
void Trap_wrapper(const float a, const float b, const int n, float* trap_p,
                  const int blk_ct, const int th_per_blk) {
    /* trap_p storage allocated in main with cudaMallocManaged */
    *trap_p = 0.5*(f(a) + f(b));
    float h = (b-a)/n;
    Dev_trap<<<blk_ct, th_per_blk>>>(a, b, h, n, trap_p);
    cudaDeviceSynchronize();
    *trap_p = h*(*trap_p);
}
```

Key points

- **Private arguments:** each CUDA thread has its own stack, so formal arguments such as `h` and `trap` are private to each thread. A **pointer** to managed memory makes `*trap_p` shared and also solves the problem of returning a value from a `void` kernel.
- **Wrapper function:** `Trap_wrapper` is a function whose main purpose is to call another function, doing preparation before and extra work after the call.
- **Correct threads:** assumes `blk_ct * th_per_blk` >= number of trapezoids; thread 0 and threads with `my_i > n-1` skip the body.
- **atomicAdd:** an operation is **atomic** if it appears to other threads as indivisible. Ordinary addition is several machine instructions, so `*trap_p += my_trap` is a race condition. `atomicAdd(float* float_p, float val)` atomically adds `val` to `*float_p` and returns the old value.

Performance

With n = 2^20 = 1,048,576 trapezoids, f(x) = x^2 + 1, a = -3, b = 3, using 1024 blocks of 1024 threads (mean run-times in ms):

ARM Cortex-A15 (serial)	Nvidia GK20A (1 SM, 192 SPs)	Intel Core i7 (serial)	GeForce GTX Titan X (24 SMs, 3072 SPs)
33.6	20.7	4.48	3.08

The Titan X is only about 45% faster than one Intel core, because `atomicAdd` **serialises** the updates to `*trap_p`.

Improvement: tree-structured sum

With basic atomic sums, `t` threads need a sequence of `t` additions; a **tree-structured sum** needs only `floor(log2 t) + 1` (1000 threads: 1000 vs 11; 1,000,000 threads: 21). In CUDA it is done with **warp shuffles** (compute capability >= 3, threads in a warp read each other's registers) or with **shared memory** (compute capability < 3).

Conclusion

In the CUDA trapezoidal rule every thread evaluates one `f(x_i)` and adds it into a shared result with `atomicAdd`; the host initialises, launches the kernel, synchronises and multiplies by `h`.

UNIT 5 - GPU Programming with CUDA

Q. 5 Explain the history of CUDA. [10 Marks]

Introduction

CUDA is the software platform that Nvidia developed for general-purpose programming of its GPUs. Its history is part of the history of GPUs and GPGPU.

Stage 1: Birth of GPUs (late 1990s - early 2000s)

- The industry responded to the demand for **highly realistic computer video games and video animations** by developing extremely powerful **graphics processing units (GPUs)**.
- GPUs were designed to improve the performance of programs that need to **render many detailed images**.

Stage 2: GPGPU (early 2000s)

- Programmers who did not specialise in graphics tried to use this computational power for **general problems such as searching and sorting**. This became known as **General Purpose computing on GPUs (GPGPU)**.
- The GPUs of the time could be programmed only with **graphics APIs**, such as **Direct3D and OpenGL**. Algorithms had to be reformulated using **vertices, triangles and pixels**, which made early GPGPU programs very complex.

Stage 3: High-level APIs

- Several groups started developing **languages and compilers** that allowed general algorithms to be written in APIs that **resemble conventional high-level languages** for CPUs.
- This led to several APIs for general-purpose GPU programming; the most widely used are **CUDA** and **OpenCL**.
- **CUDA was developed for Nvidia GPUs. OpenCL** was designed to be portable across arbitrary GPUs and other processors (FPGAs, DSPs).

Stage 4: CUDA

- CUDA was originally an acronym for **"Compute Unified Device Architecture"**, meant to suggest a single interface for programming both CPU and GPU. Nvidia has since decided that CUDA is **not an acronym**; it is simply the name of an API for GPGPU programming.
- Language-specific CUDA APIs exist for **C, C++, Fortran, Python and Java**. **CUDA C** is compiled by `nvcc`, a modified C++ compiler.
- CUDA is not just a library: it needs a special compiler that generates code for both the **host CPU** and the **device GPU**.
- Since it runs only on Nvidia GPUs it requires **relatively modest setup**, so it is used in the textbook instead of OpenCL.

Stage 5: Evolution of features and hardware

Item	Development
Unified memory	First available in **CUDA 6.0**: host and device can use the same pointers (`cudaMallocManaged`); needs compute capability >= 3.0
Synchronising threads of different blocks	Became possible with **CUDA 9** and the **Pascal** processor
Warp shuffle functions	Allow threads of a warp to read each other's registers (compute capability >= 3)
Compute capability	Version number `a.b` of a GPU; major numbers 1, 2, 3, 5, 6, 7, 8 (no 4). **CUDA no longer supports capability < 3**
Versions of CUDA API	Do **not** correspond to the compute capabilities of GPUs

Nvidia GPU microarchitectures

Name	Tesla	Fermi	Kepler	Maxwell	Pascal	Volta	Turing	Ampere
Compute capability	1.b	2.b	3.b	5.b	6.b	7.0	7.5	8.0

Nvidia also uses **Tesla** as the name for its products targeting GPGPU.

Why CUDA / heterogeneous computing became important

From about 1986 to 2003 single-thread CPU performance grew by more than **50% per year**, but from 2015 to 2017 it grew by less than **4% per year**. Programmers therefore look to other processors such as GPUs, and CUDA lets them program CPU-GPU (heterogeneous) systems.

Conclusion

CUDA grew out of the need to use GPUs, first built for graphics, for general computation without graphics APIs. It has since become a widely used API with unified memory and many architectures.

UNIT 5 - GPU Programming with CUDA

Q. 6 Write a CUDA C program to implement vector addition using parallel threads. [10 Marks]

Aim

To add two n-element vectors `x` and `y` into `z` using CUDA threads. The problem is **data-parallel** and **embarrassingly parallel**: a kernel starts at least `n` threads and thread `i` computes `z[i] = x[i] + y[i]`. `float` arrays are used because GPUs have more 32-bit than 64-bit floating-point units.

Algorithm

1. Get `n`, number of blocks `blk_ct` and threads per block `th_per_blk` (`Get_args`); check `n <= blk_ct * th_per_blk`.
2. Allocate `x, y, z` with `cudaMallocManaged` (valid on host and device) and `cz` with `malloc` (host only, for checking).
3. Initialise `x` and `y` on the host (read or random).
4. Start the kernel `Vec_add<<<blk_ct, th_per_blk>>>(x, y, z, n)`; each thread computes its global rank and adds one pair of elements.
5. `cudaDeviceSynchronize()`; compare with serial result `cz` using the two-norm; free memory.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda.h>

__global__ void Vec_add(const float x[], const float y[], float z[], const int n) {
    int my_elt = blockDim.x * blockIdx.x + threadIdx.x;   /* global rank */
    if (my_elt < n)                       /* total threads may be > n */
        z[my_elt] = x[my_elt] + y[my_elt];
}

void Serial_vec_add(const float x[], const float y[], float cz[], const int n) {
    for (int i = 0; i < n; i++) cz[i] = x[i] + y[i];
}
double Two_norm_diff(const float z[], const float cz[], const int n) {
    double diff, sum = 0.0;
    for (int i = 0; i < n; i++) { diff = z[i] - cz[i]; sum += diff*diff; }
    return sqrt(sum);
}

int main(int argc, char* argv[]) {
    int n, th_per_blk, blk_ct;
    float *x, *y, *z, *cz;
    double diff_norm;

    n          = strtol(argv[1], NULL, 10);      /* Get_args: vector size */
    blk_ct     = strtol(argv[2], NULL, 10);      /* number of blocks      */
    th_per_blk = strtol(argv[3], NULL, 10);      /* threads per block     */

    cudaMallocManaged(&x, n*sizeof(float));      /* Allocate_vectors */
    cudaMallocManaged(&y, n*sizeof(float));
    cudaMallocManaged(&z, n*sizeof(float));
    cz = (float*) malloc(n*sizeof(float));       /* host only */

    for (int i = 0; i < n; i++) {                /* Init_vectors */
        x[i] = random()/((float) RAND_MAX);
        y[i] = random()/((float) RAND_MAX);
    }

    Vec_add<<<blk_ct, th_per_blk>>>(x, y, z, n); /* invoke kernel      */
    cudaDeviceSynchronize();                     /* wait for the GPU   */

    Serial_vec_add(x, y, cz, n);                 /* check correctness  */
    diff_norm = Two_norm_diff(z, cz, n);
    printf("Two-norm of difference between host and device = %e\n", diff_norm);

    cudaFree(x); cudaFree(y); cudaFree(z);       /* Free_vectors */
    free(cz);
    return 0;
}
```

Compile and run: `$ nvcc -o cuda_vec_add cuda_vec_add.cu` then `$ ./cuda_vec_add 1000 4 256`

Explanation

- **Kernel:** `__global__`, return type `void`. Global rank `my_elt = blockDim.x*blockIdx.x + threadIdx.x`; with 4 blocks of 5 threads the ranks are 0 to 19 (block 0: 0-4, block 1: 5-9, ...). This is the serial `for` loop with each iteration given to a different thread.
- `my_elt < n` **check:** the number of threads may exceed `n`. For n = 997 and 4 blocks of 256 threads (1024), the last 27 threads skip the addition.
- `cudaMallocManaged`**:** unified memory; pointers valid on both host and device. Needs compute capability >= 3.0; for capability < 6.0 the host cannot access it while a kernel runs.
- **Asynchronous kernel:** the host returns immediately after `<<<>>>`, so `cudaDeviceSynchronize()` is needed before using `z`.
- **Checking:** `Two_norm_diff` computes the square root of the sum of squared differences between `z` and `cz`; it should be about 0.
- Memory from `cudaMallocManaged` is freed with `cudaFree`; `cz` with `free`.

Without unified memory (explicit transfers)

The kernel is unchanged. Use host arrays `hx, hy, hz` (`malloc`) and device arrays `dx, dy, dz` (`cudaMalloc`):

```
cudaMemcpy(dx, hx, n*sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(dy, hy, n*sizeof(float), cudaMemcpyHostToDevice);
Vec_add<<<blk_ct, th_per_blk>>>(dx, dy, dz, n);
cudaMemcpy(hz, dz, n*sizeof(float), cudaMemcpyDeviceToHost);  /* waits for kernel */
```

Conclusion

In CUDA vector addition, every thread finds its global index from `blockIdx`, `blockDim` and `threadIdx` and adds one element pair, so the loop of n additions is done by n parallel threads.